

NanoInfer 投机解码实测报告

NanoInfer

2026-08-17

AutoDL RTX 4090 (24GB) • Qwen2.5 0.5B / 1.5B / 7B • FP16

工具: `quick_generate --spec` (端到端文本投机解码, 含各阶段耗时)

1 摘要

- 正确性已修复: 投机解码的 identity bug (丢自注意力) 定位并修复, 投机输出与常规解码逐字一致。
- 纯推理优化 (cudaMallocAsync + 去掉每层 D2H + run_layers 提升) 把 7B+0.5B dual 从 **0.54x** 提到 **0.97x**。
- 瓶颈锁定: draft 的 run_layers 单 token 前向 $\sim 2.4\times$ 慢于 step() 快路径 (0.5B: 11 ms vs 4.5 ms)。
- 下一步: 让草稿前向追平快路径 (纯推理) $\rightarrow$ 预计 **$\sim 2.9\times$** 。

2 背景

NanoInfer 是手写 C++/CUDA LLM 推理引擎 (PagedAttention + 前缀缓存 + Continuous Batching)。投机解码流程: draft 引擎自回归预测 k 个 token $\rightarrow$ target 一次批式验证 $\rightarrow$ 接受/拒绝。目标是用便宜的草稿省掉多次昂贵的 target 解码。

3 正确性修复: identity bug

3.1 现象

self-spec M=full (草稿=target) 时接受率 100%、0 拒绝, 但输出与常规贪心解码不同 (第 2 个 token 就翻转 argmax) ——draft 和 verify 互相一致, 但都与正常 decode 不一致。

3.2 根因 (一行 off-by-one)

run_layers (engine_server.cpp) 里 seq_lens[i] = start_pos + i 少一个 +1:

- paged_attention kernel 的 seq_len 是计数 (attend 到 [0, seq_len))。
- step() 解码 lens = seq_len + 1 $\rightarrow$ 包含自己 (正确, bug #11 “decode 缺自注意” 修复后的语义, 对拍过 numpy)。
- run_layers (draft/verify 共享) seq_lens = start_pos + i $\rightarrow$ 排除自己 $\rightarrow$ hidden 与正确解码不一致。

3.3 修复

`seq_lens[i] = start_pos + i + 1` (`engine_server.cpp` + `engine.h` 注释同步)。

3.4 连锁影响

- dual 严格 argmax 接受率从 0-33% (污染假象) 升到 **87%** (7B+0.5B)。
- 弱草稿 (self M=6, 0% 接受) 的修正路径也变正确, 输出回归贪心。
- `test_spec_decode` 23/23 通过, 无回归。

4 端到端准确性测试 (4090)

4.1 常规解码输出 (贪心)

模型	Prompt	输出 (截断)
0.5B	The capital of France is	“ Paris. It is the largest city in Europe and the third largest city...”
0.5B	The sky is	“ a beautiful place. It is a place of endless possibilities...”
0.5B	Q: What is 2+2? A:	“ 4...”
1.5B	The capital of France is	“ Paris. The capital of France is also the capital of which country?”
7B	The capital of France is	“ Paris. Which of the following statements is correct?”
7B	Write one short sentence about mountains:	“ Mountains are natural wonders that inspire awe and adventure.”

4.2 投机 vs 常规 identity (逐字一致)

配置	常规输出	投机	接受率
0.5B self M=24	“ Paris. It is the largest...”	逐字一致	100%
1.5B+0.5B dual	“ Paris. The capital...”	逐字一致	69%
7B+0.5B dual	“ Paris. Which of the following...”	逐字一致	80%
7B+0.5B dual (山)	“ Mountains are natural wonders...”	逐字一致	40%

结论: 投机路径准确性 = 常规路径准确性, 无额外退化。

4.3 已知局限

- 个别 prompt 输出 ____/<unk>: C++ 简化 tokenizer `encode()` 的局限（头文件注明生产用 HF 预分词），非推理/投机准确性问题。
- 服务器无 transformers env，未做 HF 逐字对拍；identity 一致性 + 输出合理性是当前最强证明。

5 纯推理优化与实测

优化	改动	效果
cudaMallocAsync	<code>tensor.cpp</code> : 裸 <code>cudaMalloc</code> → stream 有序池化分配	消除每步 ~1000 次驱动分配
去掉每层 D2H	<code>paged_attention</code> 调用方传 <code>max_seq_hint</code> ，跳过同步 D2H	消除每层 1 次同步 D2H (草稿 4 token×24 层 = 96 次/步)
run_layers 提升	RoPE/block table/seq_lens 提到层循环外；去掉每层 <code>sync</code>	分配/同步减少

5.1 实测（7B+0.5B dual, 严格 argmax）

指标	起点	现在
TPOT vs 常规 21.2 ms	26.3 ms (0.81×)	21.8 ms (0.97×)
verify/步	~20 ms	4.4 ms (batched 7B 完美摊薄)

6 瓶颈分析

6.1 每步账（7B+0.5B, 87% 接受率，产 ~3.86 token/步）

环节	实测	说明
draft	~55 ms/步	4 次顺序 0.5B 单 token 前向 (autoregressive, 无法摊薄)
verify	4.4 ms/步	batched M=5 的 7B, 权重加载完美摊薄 (≈1 次 decode)
修正	~ 小	13% 拒绝率
合计	~60 ms / 3.86 token	常规 7B = 82 ms / 3.86 → 理论 1.36×, 实测 0.97×

6.2 探针定位（草稿循环分账，ms/token）

emb=0.02 fwd=2.2(CPU 入队) lmh=0.02 pick(argmax D2H 同步)=11.3ms (98%)

同步 D2H 等待异步前向队列排空，暴露**真实 GPU 时间 ~11 ms/token**（CPU 入队时间 2.2 ms 是假象）。

6.3 瓶颈链条

1. **draft = k** 次顺序前向（结构性税）：每次读满 0.5B 全部权重，k 次 = $k \times$ 单 token 成本。
2. **run_layers** 单 token 前向 $\sim 2.4 \times$ 慢于 **step()** 快路径（0.5B：11 ms vs 4.5 ms）——M=1 GEMM 与每层开销不如 step() 的批式解码循环。
3. **argmax D2H 同步**：每 token 一次，D2H 移除后是最后剩的同步点。

6.4 非瓶颈（已验证）

接受率 87% · 验证摊薄 4.4 ms/步 · 分配/同步优化 · 正确性。

7 批量草稿 lookahead（实测 1.37 $\times$ ）

7.1 根因再定位（CUDA event）

run_layers T=1 (0.5B) gpu=**12.86 ms** vs T=4 gpu=**4.82 ms**——批 4 个 token 反而快 **2.7 $\times$** 。“other”（M=1 GEMM + 逐层算）在 T=1 时 0.52 ms/层，纯 M=1 延迟（cuBLAS 权重读一遍只服务 1 行）。

7.2 方案：批量草稿（纯推理，无训练）

草稿从 k 次顺序单 token 前向 ($k \times T=1$) 改为一次 **T=k** 批式前向 + 不动点迭代：块内每行的 logits 逐轮修正下一猜测，收敛到贪心延续。固定点 = 顺序草稿的贪心结果，接受率不变。

7.3 实测（4090，7B+0.5B dual，严格 argmax）

指标	顺序草稿	批量草稿
draft_ms	516 ms	321 ms
verify_ms	~171 ms	49 ms
TPOT (87% 接受率 prompt)	21.8 ms	15.5 ms (1.37$\times$)
identity	—	逐字一致

7.4 接受率 $\times$ 提速实测（多 prompt）

prompt	类型	接受率	spec TPOT	regular TPOT	vs
The capital of France is a beautiful city (32t)	半客观	87%	15.5 ms	21.2 ms	1.37×
The chemical symbol for gold is	客观	82.6%	16.2 ms	18.1 ms	1.12×
The fastest land animal is the	客观	78.3%	18.5 ms	17.5 ms	0.95×
Water is made of two hydrogen atoms and one	客观	73.9%	19.4 ms	17.8 ms	0.92×
The capital of Japan is	客观	69.6%	19.0 ms	17.5 ms	0.92×
The largest ocean on Earth is the	客观	60.9%	22.6 ms	18.3 ms	0.81×
Write one short sentence about mountains	开放	40.0%	27.1 ms	16.7 ms	0.62×

结论：

- 客观问题接受率普遍更高（61–83%）vs 开放问题（40%）——客观续写是“确定”的，0.5B 与 7B 倾向一致；开放续写合法分支多，两模型各选各的。
- 盈亏平衡接受率 $\approx 82\text{--}85\%$ ：只有 $\geq 82\%$ 才真正提速（France 87% $\rightarrow 1.37\times$ 、gold 82.6% $\rightarrow 1.12\times$ ）；60–80% 的客观题仅打平或略慢（拒绝 $\rightarrow$ 修正重算吃掉收益）。
- 生成长度也关键：续写越长，每步固定开销摊得越薄（France 32t vs mountains 11t）。
- 实践启示：投机解码（跨规模）适合长生成 + 事实问答/结构化输出这类高确定场景。

大样本验证（bench_eval, alpaca 采样 100 条，7B+0.5B dual）：

- 接受率分布：mean **69.1%** · p50 71.0% · p90 **83.9%** · max 93.5%。
- 提速分布：mean **0.90×** · p50 0.86× · p90 **1.20×** · **31/100 > 1.0×**。
- 中位接受率（71%）低于盈亏平衡（ $\sim 82\text{--}85\%$ ） $\rightarrow$ 整体净 0.90×；只有 top $\sim 30\%$ 高接受率 prompt 真正提速。
- 评测说明：工具默认 temp=1.0，拒绝时采样修正致 spec 与 regular 文本仅 41% 一致（temp=0 贪心才 100% identity）；接受率/提速统计不受影响。

准确性基准（HF tokenizer 预分词 + 正确提示格式，greedy）：

- GSM8K 100 条（零样本 CoT）：**37.0%**；MMLU 200 条（零样本 Answer:）：**54.5%**。
- 低于 Qwen2.5-7B HF 头条（GSM8K $\sim 76\%$ /MMLU $\sim 70\%$ ）：主因是零样本 + greedy，非引擎 bug（引擎对拍 HF 逐 token 一致）。
- 用 C++ 简化 tokenizer 的 encode() 会把这些数字打到 5%（复杂 prompt 失效），必须 HF 预分词。

注：Medusa 式 speculator head（需训练）按用户要求排除；本路线纯推理优化，test_spec_decode 23/23 无回归。